कंप्यूटर सॉफ़्टवेयर अभियांत्रिकी

सॉफ़्टवेयर इंजीनियरिंग को हिंदी में "कंप्यूटर सॉफ़्टवेयर अभियांत्रिकी" या "सॉफ़्टवेयर अभियांत्रिकी" कहा जाता है। यह एक विज्ञान है जो सॉफ़्टवेयर के विकास, डिज़ाइन, निर्माण और प्रबंधन के सिद्धांतों, तकनीकों और उपकरणों का अध्ययन करती है। सॉफ़्टवेयर इंजीनियरिंग में, सॉफ़्टवेयर की विभिन्न पहलुओं को ध्यान में रखकर उच्च गुणवत्ता वाले सॉफ़्टवेयर प्रोजेक्ट्स विकसित किए जाते हैं।

सॉफ्टवेयर इंजीनियर का मुख्य उद्देश्य सुरक्षित, दुरुस्त, और उपयोगकर्ता की आवश्यकताओं को पूरा करने वाला सॉफ्टवेयर बनाना होता है

सॉफ्टवेयर इंजीनियरिंग एक प्रक्रिया है जिसमें सॉफ्टवेयर के विकास, निर्माण, और परिचालन के लिए विज्ञान, तकनीकी ज्ञान, और अनुभव का उपयोग किया जाता है। इसमें सॉफ्टवेयर की आवश्यकताओं को समझना, उन्हें प्रोग्रामिंग भाषाओं में व्याख्यात्रा करना, सॉफ्टवेयर डिजाइन और आर्किटेक्चर तैयार करना, और सॉफ्टवेयर को परीक्षण, अनुकूलन, और दुरुस्त करना शामिल होता है।

सॉफ्टवेयर इंजीनियरिंग के लिए कुछ मुख्य चरण होते हैं: विश्लेषण, योजना बनाना, विकास, परीक्षण और स्थानांतरण। इन चरणों के दौरान, विकसित सॉफ्टवेयर के गुणवत्ता, प्रदर्शन, सुरक्षा और स्थायित्व पर ध्यान दिया जाता है। सॉफ्टवेयर इंजीनियर विभिन्न प्रोग्रामिंग भाषाओं, टूल्स, फ्रेमवर्क, और मेथडों का उपयोग करते हुए सॉफ्टवेयर का निर्माण करता है।

कंप्यूटर सॉफ़्टवेयर अभियांत्रिकी के चरण निम्नलिखित होते हैं:

1. आवश्यकता विश्लेषण: इस चरण में, सॉफ़्टवेयर इंजीनियरों को उपयोगकर्ताओं द्वारा की गई आवश्यकताओं को समझना होता है। इसमें आवश्यकताओं का विश्लेषण, विशेषताओं की परिभाषा, और प्रमुख लक्षणों की निर्धारण शामिल होता है।

2. निर्माण योजना: यह चरण निर्माण योजना का तैयारी और प्रबंधन के लिए समर्पित होता है। यह योजना कार्य को व्यवस्थित करने, संसाधनों का प्रबंधन करने, और प्रोजेक्ट की अवधि, लागत, और आवश्यक प्रतिस्पर्धा का मूल्यांकन करने के लिए उपयोगी होता है।

3. डिज़ाइन: डिज़ाइन चरण में, सॉफ़्टवेयर की संरचना, अंतरफलक, और उपयोगकर्ता इंटरफ़ेस को तैयार किया जाता है। यह चरण सॉफ़्टवेयर के आर्किटेक्चर, मॉड्यूल, डेटा संरचना, अल्गोरिदम्स, और सुरक्षा के लिए निर्देशक होता है।

4. प्रोग्रामिंग: इस चरण में, सॉफ़्टवेयर के विभिन्न घटकों को विकसित करने और लिखने के लिए कंप्यूटर प्रोग्रामिंग भाषाओं का उपयोग किया जाता है। यहां प्रोग्रामर सॉफ़्टवेयर कोड को लिखते हैं और उन्हें टेस्ट और डिबग करते हैं।

5. टेस्टिंग: यह चरण सॉफ़्टवेयर की परीक्षण को समर्पित होता है। इसमें विभिन्न परीक्षण तकनीकों का उपयोग करके सॉफ़्टवेयर को परीक्षण किया जाता है और बग या त्रुटियों को ध्यान में रखा जाता है।

6. दृढ़ीकरण और वितरण: इस चरण में, सॉफ़्टवेयर को उपयोगकर्ताओं तक पहुंचाने के लिए उसे दृढ़ीकृत किया और वितरित किया जाता है। सॉफ़्टवेयर की स्थापना, उपयोगकर्ता प्रशिक्षण, और समर्थन कार्य इस चरण में सम्पन्न होते हैं।

ये सभी चरण संगठित तरीके से सॉफ़्टवेयर अभियांत्रिकी प्रक्रिया के हिस्से होते हैं और उच्च गुणवत्ता वाले सॉफ़्टवेयर प्रोजेक्ट्स को विकसित करने में मदद करते हैं।

IEEE (Institute of Electrical and Electronics Engineers) सॉफ़्टवेयर अभियांत्रिकी को इस प्रकार परिभाषित करता है:

"सॉफ़्टवेयर अभियांत्रिकी, विज्ञान, और कला है जिसमें प्रकृति की विभिन्न धाराओं, सिद्धांतों, और विधियों का उपयोग करके सॉफ़्टवेयर की उत्पत्ति, विकास, और बचाव की प्रक्रिया को व्यवस्थित किया जाता है।"

इस परिभाषा के अनुसार, सॉफ़्टवेयर अभियांत्रिकी का उद्देश्य सॉफ़्टवेयर के उत्पादन और विकास के विभिन्न पहलुओं को व्यवस्थित करना है, जो नियमित सिद्धांतों, तकनीकों, और विधियों के आधार पर प्राप्त किए जाते हैं।

सॉफ़्टवेयर विकास (Software Evolution)

सॉफ़्टवेयर विकास (Software Evolution) एक प्रक्रिया है जिसमें सॉफ़्टवेयर का निर्माण, परिवर्तन और अद्यतन किया जाता है ताकि वह समय के साथ बदल जाने वाले आवश्यकताओं और परिस्थितियों को सामर्थ्यपूर्ण रूप से पूरा कर सके। सॉफ़्टवेयर विकास का मुख्य उद्देश्य संशोधन, नवीनीकरण और उन्नति है ताकि सॉफ़्टवेयर प्रोडक्ट प्राप्तकर्ताओं की आवश्यकताओं को पूरा कर सके और प्रदर्शन और सुरक्षा के मानकों को पालन कर सके।

सॉफ़्टवेयर विकास की प्रक्रिया चरणों में विभाजित होती है। इन चरणों का विवरण निम्नानुसार है:

1. आवश्यकता विश्लेषण: इस चरण में, सॉफ़्टवेयर विकास की आवश्यकता को समझा जाता है। इसमें उपयोगकर्ताओं की आवश्यकताओं और अनुमानित उपयोग के साथ सॉफ़्टवेयर के लक्ष्यों की परिभाषा की जाती है।
2. निर्माण योजना: निर्माण योजना चरण में, सॉफ़्टवेयर विकास के लिए योजना तैयार की जाती है। यहां परियोजना की व्यवस्था, संसाधन विनियोजन और समय सारणी तैयार की जाती है।
3. संगठन: इस चरण में, सॉफ़्टवेयर के घटकों को संगठित किया जाता है। यहां सॉफ़्टवेयर की संरचना, आपूर्ति लाइफसाइकल, मॉड्यूलरिटी और पुनःउपयोग की संरचना तैयार की जाती है।
4. विकास: विकास चरण में, सॉफ़्टवेयर का निर्माण होता है। प्रोग्रामिंग, विनिर्माण, परीक्षण और डिबगिंग के माध्यम से सॉफ़्टवेयर को विकसित किया जाता है।

5. परिवर्तन: सॉफ़्टवेयर विकास का यह चरण सॉफ़्टवेयर में परिवर्तन और सुधार के लिए समर्पित होता है। इसमें आवश्यक बग निर्देशिका, प्रोग्राम कोड की संशोधन, और नवीनीकरण कार्य होते हैं।

6. उपयोग और अनुरक्षण: इस चरण में, सॉफ़्टवेयर को उपयोग में लाया जाता है और उसका अनुरक्षण किया जाता है। इसमें समर्थन सेवाएं, त्रुटि संशोधन, और निरंतर मूल्यांकन शामिल होते हैं।

सॉफ़्टवेयर विकास की यह चरण संगठित तरीके से सॉफ़्टवेयर को उन्नत और उपयोगी बनाने में मदद करता है। यह सुनिश्चित करता है कि सॉफ़्टवेयर आवश्यकताओं को सामर्थ्यपूर्ण रूप से पूरा करता है और नवीनतम तकनीकी प्रगति को ध्यान में रखता है।

सॉफ़्टवेयर विकास के दौरान, कुछ महत्वपूर्ण सॉफ़्टवेयर विकास कानूनों (Software Evolution Laws) का पालन किया जाता है जो सॉफ़्टवेयर के बदलते प्रकार और प्रगति को संकेत करते हैं। यहां कुछ प्रमुख सॉफ़्टवेयर विकास कानूनों का हिंदी में वर्णन किया गया है:

1. कानून परिवर्तन का: इस कानून के अनुसार, सॉफ़्टवेयर बदलते समय में परिवर्तित होता है। यह सॉफ़्टवेयर में नए विशेषताओं, सुधारों और बगों को प्रवेश करने का संकेत देता है।
2. कानून संवर्धन का: यह कानून सॉफ़्टवेयर की सामरिक प्रगति को दर्शाता है। सॉफ़्टवेयर विकास में नई तकनीकों, मेथडों और प्रगति के साथ एक सुधार की प्रक्रिया शामिल होती है।
3. कानून एकांत का: इस कानून के तहत, सॉफ़्टवेयर अभियांत्रिकी के तहत एकांत महत्वपूर्ण है। यह श्रोत, प्रोजेक्ट, और संगठन के सभी हिस्सों के बीच संचार और सहयोग को प्रोत्साहित करता है।
4. कानून संवाद का: यह कानून सॉफ़्टवेयर विकास में विभिन्न हिस्सेदारों के साथ संवाद को प्रोत्साहित करता है। यह सहयोग, टीमवर्क, और संघटन की आवश्यकता को उजागर करता है ताकि सॉफ़्टवेयर का विकास सुगम और सफल हो सके।

ये सॉफ़्टवेयर विकास के कुछ प्रमुख कानून हैं, जो सॉफ़्टवेयर के विकास की प्रक्रिया में महत्वपूर्ण भूमिका निभाते हैं।

Lehman's Laws of Software Evolution

लेहमन ने सॉफ़्टवेयर विकास को विभिन्न तरीकों से श्रेणीबद्ध किया है। यहां लेहमन द्वारा उपयोग की गई कुछ अलग-अलग श्रेणीकरणों का वर्णन है:

कंप्यूटर सॉफ़्टवेयर अभियांत्रिकी

1. एस-कर्व मॉडल: लेहमन ने एस-कर्व मॉडल का प्रस्तावित किया था जो समय के साथ सॉफ़्टवेयर सिस्टम के विकास और पतन को दर्शाता है। यह मॉडल प्रारंभिक धीमी वृद्धि, तेज वृद्धि दौरान का चरण और अंततः एक सतह पर पहुंचने को दिखाता है।
2. विशेषताओं पर आधारित श्रेणीकरण: लेहमन ने सॉफ़्टवेयर सिस्टम को उनकी प्राकृतिक विशेषताओं के आधार पर श्रेणीबद्ध किया, जैसे कि उनकी संचालन प्रोफ़ाइल, आकार, जटिलता, और महत्वपूर्णता। यह श्रेणीकरण विभिन्न प्रकार के सॉफ़्टवेयर सिस्टम के साथ जुड़े अद्वितीय चुनौतियों और गतिविधियों को समझने में मदद करता है।
3. सिस्टम प्रकार पर आधारित श्रेणीकरण: लेहमन ने सॉफ़्टवेयर सिस्टम को उनके डोमेन या एप्लिकेशन के आधार पर विभाजित किया, जैसे कि इंबेडेड सिस्टम, रियल-टाइम सिस्टम, व्यावसायिक एप्लिकेशन, वैज्ञानिक सॉफ़्टवेयर, आदि। प्रत्येक सिस्टम प्रकार में अपनी विशेष विशेषताएं और विकास पैटर्न हो सकते हैं।
4. रखरखाव और विकास पर आधारित श्रेणीकरण: लेहमन ने रखरखाव (सॉफ़्टवेयर को संचालित रखना) और विकास (नई सुविधाओं को जोड़ना, क्षमता को बढ़ाना) को अलग-अलग श्रेणीबद्ध करने की महत्वपूर्णता पर बल दिया। यह श्रेणीकरण सॉफ़्टवेयर सिस्टम को बनाए रखने और विकसित करने में शामिल विभिन्न गतिविधियों और चुनौतियों को समझने में मदद करता है।
5. संगठनिक कारकों पर आधारित श्रेणीकरण: लेहमन ने सॉफ़्टवेयर विकास को श्रेणीबद्ध करते समय संगठनिक कारकों को भी महत्व दिया। इसमें विकास टीम की विशेषज्ञता, संचार प्रक्रियाएं, विकास मेथडोलॉजी, और विकास संगठन की स्थिरता जैसे कारक शामिल होते हैं। ये कारक सॉफ़्टवेयर विकास और बदलती आवश्यकताओं को प्रभावित करते हैं।

ये श्रेणीकरण सॉफ़्टवेयर विकास को समझने और प्रबंधित करने के लिए विभिन्न दृष्टिकोण और ढांचे प्रदान करते हैं, जिससे अनुसंधानकर्ताओं और व्यवहारिकों को सॉफ़्टवेयर सिस्टम को प्रभावी रूप से विश्लेषण और प्रबंधित करने में सहायता मिलती है।

एस-कर्व मॉडल (S-Curve Model) लेहमन द्वारा प्रस्तावित एक मॉडल है जो सॉफ़्टवेयर विकास की प्रक्रिया को वर्णित करता है। इस मॉडल का उपयोग विकास की गति, प्रगति और श्रम का मूल्यांकन करने के लिए किया जाता है। यह मॉडल विकास की चरम गति, प्रारंभिक मेहनत, गतिशीलता और स्थिरता के चरणों को दर्शाता है।

एस-कर्व मॉडल को एक साइनोइड कर्व के रूप में प्रस्तुत किया जाता है, जो गतिशीलता और प्रगति के दृष्टिकोण से संबंधित होता है। मॉडल में तीन मुख्य चरण होते हैं:

1. अवसरवाद (Opportunity Phase): यह चरण विकास की प्रारंभिक दौर है, जहां प्रोजेक्ट की शुरुआती मेहनत होती है और प्रगति धीमी होती है।
2. विकास गति (Rapid Growth Phase): यह चरण विकास की मुख्य चरम गति है, जहां प्रोजेक्ट की प्रगति सबसे तेज़ होती है। इसमें संसाधनों का अधिक उपयोग होता है और सॉफ़्टवेयर की संरचना और फ़ंक्शनलिटी का विकास होता है।
3. स्थिरता (Maturity Phase): यह चरण विकास की स्थिरता चरण है, जहां प्रोजेक्ट की प्रगति स्थिर होती है और उपयोगकर्ता को पूर्ण उपयोग के लिए सॉफ़्टवेयर मिलता है। इस चरण में अधिकतर मेहनत अवसरवाद के समान होती है।

कंप्यूटर सॉफ़्टवेयर अभियांत्रिकी

एस-कर्व मॉडल विकास की अवधि, प्रगति और श्रम के प्रतिसाद में एक निर्दिष्ट प्रक्रिया को दर्शाता है। इस मॉडल का उपयोग सॉफ़्टवेयर विकास के निर्माण, मूल्यांकन और प्रबंधन में किया जाता है।

लेहमन द्वारा विशेषताओं पर आधारित श्रेणीकरण करने पर सॉफ़्टवेयर को विभिन्न श्रेणियों में समूहीकृत किया जाता है। इसके अनुसार, सॉफ़्टवेयर को निम्नलिखित विशेषताओं के आधार पर वर्गीकृत किया जा सकता है:

1. आकार: सॉफ़्टवेयर की आकार के आधार पर श्रेणीकृत किया जाता है, जैसे कि छोटे, मध्यम और बड़े स्केल का सॉफ़्टवेयर।
2. जटिलता: सॉफ़्टवेयर की जटिलता के आधार पर श्रेणीकृत किया जाता है, जैसे कि साधारण, मध्यम और विशेषज्ञ स्तर का सॉफ़्टवेयर।
3. परियोजना कठिनाई: सॉफ़्टवेयर परियोजना की कठिनाई के आधार पर श्रेणीकृत किया जाता है, जैसे कि सरल, मध्यम और अधिक कठिन परियोजना।
4. परियोजना की महत्ता: सॉफ़्टवेयर परियोजना की महत्ता या उद्देश्य के आधार पर श्रेणीकृत किया जाता है, जैसे कि सामान्य, महत्त्वपूर्ण और अत्यधिक महत्त्वपूर्ण सॉफ़्टवेयर।
5. प्रयोग क्षेत्र: सॉफ़्टवेयर के प्रयोग क्षेत्र के आधार पर श्रेणीकृत किया जाता है, जैसे कि व्यावसायिक, मानव सेवा, विज्ञान और इंजीनियरिंग, आदि।
6. प्रयोगकर्ता बाधाएं: सॉफ़्टवेयर के प्रयोगकर्ता बाधाओं के आधार पर श्रेणीकृत किया जाता है, जैसे कि आम उपयोगकर्ता, विशेषज्ञ उपयोगकर्ता, आदि।
7. विकास मॉडल: सॉफ़्टवेयर के विकास मॉडल के आधार पर श्रेणीकृत किया जाता है, जैसे कि जलद विकास, अवकाश विकास, प्राकृतिक भ्रष्टाचार मॉडल, आदि।

ये विभिन्न श्रेणीकरण विशेषताओं के आधार पर सॉफ़्टवेयर को वर्गीकृत करने में मदद करते हैं और उसके विकास और प्रबंधन को सुविधाजनक बनाते हैं।

यहां लेहमन द्वारा सिस्टम प्रकार पर आधारित श्रेणीकरण के कुछ उदाहरण हैं:

1. इंबेडेड सिस्टम: यह श्रेणी उन सॉफ़्टवेयर सिस्टमों को सम्मिलित करती है जो उपकरण, यांत्रिक उपकरण, या संयुक्त यंत्रों के साथ संचालित होते हैं। ये सिस्टम विभिन्न क्षेत्रों में उपयोग होते हैं, जैसे कि कार, एलईडी टीवी, स्मार्टफ़ोन, मेडिकल उपकरण, आदि।
2. रियल-टाइम सिस्टम: इस श्रेणी में वे सॉफ़्टवेयर सिस्टम शामिल होते हैं जो वास्तविक समय में डेटा प्रसंस्करण, निर्धारण, और प्रतिक्रियाओं को संचालित करते हैं। उदाहरण के लिए, विमान निर्देशन सिस्टम, बिजली ग्रिड का प्रबंधन सिस्टम, विश्वसनीयता प्रमाणीकरण सिस्टम, आदि।
3. व्यावसायिक एप्लिकेशन: इस श्रेणी में व्यावसायिक उद्योगों और संगठनों के लिए संदर्भित सॉफ़्टवेयर उपयोग किए जाते हैं। इनमें वित्तीय प्रबंधन सिस्टम, व्यापार संचालन सिस्टम, ग्राहक संचालन सिस्टम, सप्लाई चेन मैनेजमेंट सिस्टम, आदि शामिल होते हैं।

4. वैज्ञानिक सॉफ़्टवेयर: इस श्रेणी में वैज्ञानिक अनुसंधान और प्रयोगशालाओं के लिए सॉफ़्टवेयर शामिल होते हैं। ये सॉफ़्टवेयर वैज्ञानिक उपयोग के लिए विशेष विशेषताएं और गतिविधियों को समर्थन करते हैं, जैसे कि डेटा विश्लेषण, मॉडलिंग और सिमुलेशन, वैज्ञानिक विज़ुअलाइज़ेशन, आदि।

यह सॉफ़्टवेयर सिस्टमों के विभिन्न प्रकारों को समझने और प्रबंधित करने में सहायता करता है और विकास और प्रबंधन में विशेषज्ञता और दृष्टिकोण प्रदान करता है।

लेहमन सॉफ़्टवेयर विकास के कुछ महत्वपूर्ण कानूनों (Lehman's Laws of Software Evolution) का अध्ययन किया गया है, जो सॉफ़्टवेयर के विकास की तरफ प्रेरित करते हैं। यहां लेहमन सॉफ़्टवेयर विकास के कुछ मुख्य कानूनों का हिंदी में वर्णन किया गया है:

---

1. पहला कानून - बदलाव का कानून (Law of Continuing Change): इस कानून के अनुसार, सॉफ़्टवेयर निरंतर बदलता रहता है। सॉफ़्टवेयर की आवश्यकताएं, विशेषताएं और परिकल्पनाएं समय के साथ परिवर्तित होती रहती हैं।

2. दूसरा कानून - ऊर्ध्वमुखी विकास का कानून (Law of Increasing Complexity): इस कानून के अनुसार, सॉफ़्टवेयर की जटिलता और विस्तार में निरंतर वृद्धि होती है। नई विशेषताएं और बग निर्देशिका सॉफ़्टवेयर को जटिल बनाते रहते हैं।

3. तीसरा कानून - स्वतःचलित विकास का कानून (Law of Self-Regulation): इस कानून के अनुसार, सॉफ़्टवेयर के विकास के दौरान स्वतःचलित संगठन होता है। यह सॉफ़्टवेयर को संशोधित करने और अपने आप को समायोजित करने की क्षमता प्रदान करता है।

4. चौथा कानून - पुराने संसाधनों का कानून (Law of Conservation of Familiarity): इस कानून के अनुसार, सॉफ़्टवेयर के विकास के दौरान पुराने संसाधनों और क्षमताओं का ध्यान रखना महत्वपूर्ण होता है। इससे पुराने संसाधनों का उपयोग बढ़ाया जा सकता है और संगठन के लिए उपयोगी हो सकता है।

ये लेहमन सॉफ़्टवेयर विकास के कुछ मुख्य कानून हैं, जो सॉफ़्टवेयर के विकास की प्रक्रिया में महत्वपूर्ण भूमिका निभाते हैं।

लेहमन के अनुसार, सॉफ़्टवेयर को निम्नलिखित प्रकारों में वर्गीकृत किया जा सकता है जो उनकी विशेषताओं और व्यवहार के आधार पर होते हैं:

1. एस-प्रकार (S-Type, सजीव): ये सॉफ़्टवेयर प्रणालियाँ उपयोगकर्ता की आवश्यकताओं को पूरा करती रहती हैं और अपेक्षाकृत स्थिर होती हैं। इन्हें मूलभूत डिज़ाइन में महत्वपूर्ण परिवर्तन के बिना तत्परता से अद्यतन और रखरखाव किया जाता है।

2. पी-प्रकार (P-Type, प्रगतिशील): ये सॉफ़्टवेयर प्रणालियाँ नियमित रूप से विकसित होती हैं और उपयोगकर्ताओं की बदलती आवश्यकताओं और अपेक्षाओं को पूरा करने के लिए संशोधन, नई सुविधाओं और उन्नयन की आवश्यकता होती है।

3. आर-प्रकार (R-Type, तेजी से बदलनेवाली): ये सॉफ़्टवेयर प्रणालियाँ तेजी से बदलती रहती हैं क्योंकि उपयोगकर्ता की आवश्यकताओं, प्रौद्योगिकी प्रगति या व्यावसायिक पर्यावरण में निरंतर परिवर्तन होता है। इन्हें आवश्यक बदलाव और पुनर्गठन के माध्यम से अपडेट किया जाता है ताकि यह बदलते आवश्यकताओं का संकेत कर सके।

4. सी-प्रकार (C-Type, सत्यापित): ये सॉफ़्टवेयर प्रणालियाँ समय के साथ अपरिवर्तित रहती हैं, आमतौर पर उपयोगकर्ता की आवश्यकताओं में स्थिरता होती है या संसाधनों की सीमाओं के कारण। इन्हें विरासती प्रणालियाँ या नियमों द्वारा नियंत्रित किया जाता है, जहां परिवर्तनों को कम से कम या नजरअंदाज किया जाता है।

5. ए-प्रकार (A-Type, आवेदनिक): ये सॉफ़्टवेयर प्रणालियाँ किसी विशेष एप्लिकेशन या क्षेत्र के लिए विकसित होती हैं। इन्हें उपयोगकर्ताओं की विशेष आवश्यकताओं को पूरा करने के लिए तैयार किया जाता है और उनका व्यापक सामान्यता और पुनर्गुणवत्ता कम हो सकता है।

ये वर्गीकरण सॉफ़्टवेयर प्रणालियों की विभिन्न विशेषताओं और गतिविधियों को समझने में मदद करते हैं और सॉफ़्टवेयर विकास और प्रबंधन प्रक्रिया का मार्गदर्शन करते हैं।

E-type software evolution laws

ए टाइप सॉफ़्टवेयर विकास के लिए लेहमन के 8 नियम हिंदी में निम्नलिखित हैं:

1. सतत परिवर्तन: सॉफ़्टवेयर सिस्टम समय के साथ सतत विकसित होते रहते हैं ताकि नए आवश्यकताओं को पूरा कर सकें और परिवर्तित होने वाले पर्यावरण के साथ समायोजित हो सकें।

2. बढ़ती जटिलता: सॉफ़्टवेयर विकसित होने के साथ-साथ उसकी जटिलता भी बढ़ती है क्योंकि नए सुविधाओं, सुधारों और परिवर्तनों को जोड़ा जाता है।

3. स्व-नियंत्रण: सॉफ़्टवेयर सिस्टम में स्व-नियंत्रण की प्राकृतिक प्रवृत्ति होती है, जिसमें परिवर्तन विकास टीम द्वारा संभाले जा सकने वाले दर पर होते हैं।

4. परिचय की संरक्षण: जटिलता को संघटित करने के लिए, सॉफ़्टवेयर विकास की प्रक्रिया में मौजूदा संरचना, घटक और इंटरफ़ेस को संरक्षित रखने की प्रवृत्ति होती है ताकि विचलितता कम हो और अव्यवस्था कम हो।

5. सतत विकास: सॉफ़्टवेयर सिस्टम में आवश्यकताओं को पूरा करने के लिए नई सुविधाएं और क्षमताएं जोड़ने के कारण, इसका सतत विकास होता है।

6. गुणवत्ता की क्षीणता: नियमित रूप से नहीं रखी जाए तो सॉफ़्टवेयर की गुणवत्ता समय के साथ क्षीण होती है, क्योंकि त्रुटियों, बगों और तकनीकी कर्ज की संचय होती है।

7. प्रतिक्रिया प्रणाली: उपयोगकर्ताओं और विकासकर्ताओं के बीच प्रतिक्रिया लूप सॉफ़्टवेयर विकास में महत्वपूर्ण भूमिका निभाती है, जो उसके भविष्य की दिशा और सुधारों को आकार देती है।

8. संगठनिक स्थिरता: विकास संगठन की स्थिरता और उसके सदस्यों की विशेषज्ञता सॉफ़्टवेयर विकास और बदलती आवश्यकताओं को प्रभावित करती है।

ये नियम, लेहमन और उनके सहयोगियों द्वारा तैयार किए गए हैं और इंबेडेड सिस्टम में सॉफ़्टवेयर विकास के पैटर्न और गतिविधियों के बारे में संदर्भ प्रदान करते हैं।